Penetration Testing & Hardening Report

Project: Secure 3-Tier Web Architecture & Penetration Testing Framework

Student: Farah Ismaeil

Executive Summary

This report documents a comprehensive penetration testing exercise conducted on a custom-built 3-tier web architecture. The lab simulates a real-world company infrastructure with Kali Linux acting as an ethical hacker and Ubuntu Server as the target system. Over three weeks, multiple critical vulnerabilities were discovered, exploited safely, documented, and then remediated. This exercise demonstrates the complete cybersecurity lifecycle: assessment, exploitation, and defensive hardening.

Phase 1: Virtual Infrastructure Realization

Objective:

Build an isolated, secure lab environment that simulates a real corporate network without risking actual systems.

Infrastructure Deployed:

Component	Specification	Purpose
Host OS	Windows with VirtualBox 7.2.8	Virtualization platform
Kali Linux	Version 2026.1 (64-bit)	Attacker machine with security tools
Ubuntu Server	Version 26.04 LTS	Target web server
Network Type	Host-Only (192.168.56.0/24)	Isolated private network
Kali IP	192.168.56.101	Attacker system
Ubuntu IP	192.168.56.102	Target system

LAMP Stack Installed:

Linux (Ubuntu 26.04): Operating system for the web server

Apache 2.4.66: Web server software listening on port 80

MySQL 8.4.8: Relational database for storing user data

PHP 8.5: Server-side scripting language for dynamic web pages

Additionally, SSH OpenSSH 10.2 was installed on port 22 for remote administration.

Database Setup:

Created database **vulnerable_db** with table **users** containing test accounts:

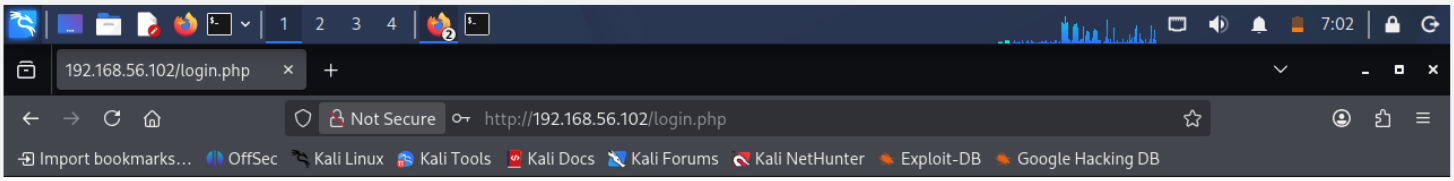
• admin / password123

• farah / farah123

MySQL user **phpuser** was created with initial full database access for web application use.

Vulnerable Web Application:

Created **login.php** - a login page intentionally written with security flaws to simulate real-world vulnerabilities. The PHP code directly concatenates user input into SQL queries without any sanitization or prepared statements, making it vulnerable to SQL injection attacks.



Phase 2: Automated Discovery Scripting

Objective:

Write custom Python scripts to discover information about the target system - mimicking real-world reconnaissance that hackers perform before attacking.

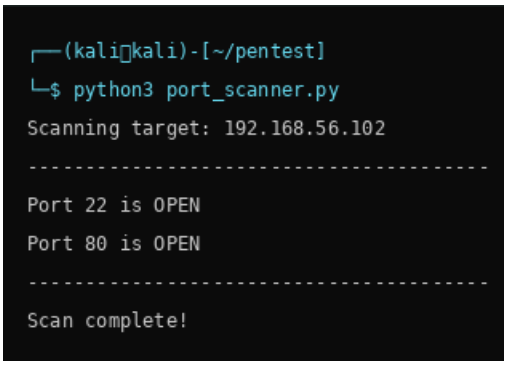
1. Port Scanner (port_scanner.py)

Purpose: Scan all 1024 ports to identify open services

Method: Attempts TCP connection to each port; records successful connections

Results: Found ports 22 (SSH) and 80 (HTTP) open; 998 ports closed

Security Implication: Only two attack surfaces available - a good security practice



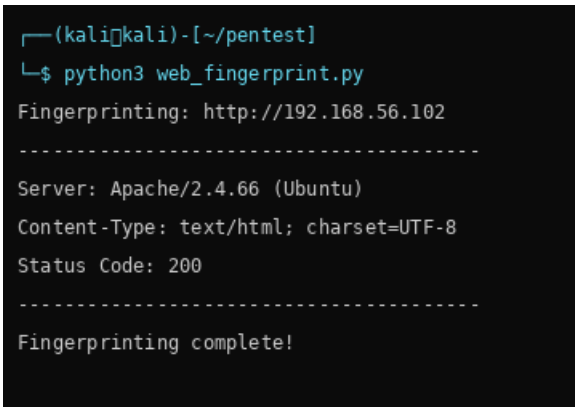
2. Web Fingerprinter (web_fingerprint.py)

Purpose: Identify exact web server software and version

Method: Reads HTTP response headers sent by the server

Results: Apache 2.4.66 on Ubuntu Linux

Security Implication: Knowing exact version allows searching for known vulnerabilities specific to that release



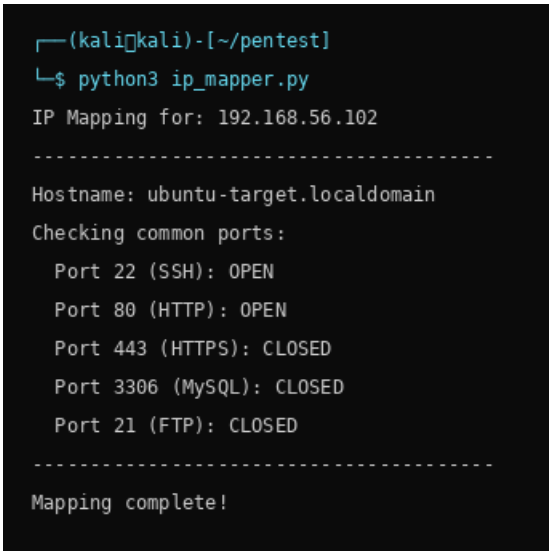
3. IP Mapper (ip_mapper.py)

Purpose: Map target network services and check common ports

Method: Attempts connections to ports 22, 80, 443, 3306, 21

Results: Only SSH (22) and HTTP (80) open; MySQL (3306) is NOT exposed to network

Security Implication: Database is properly isolated - only accessible locally via PHP



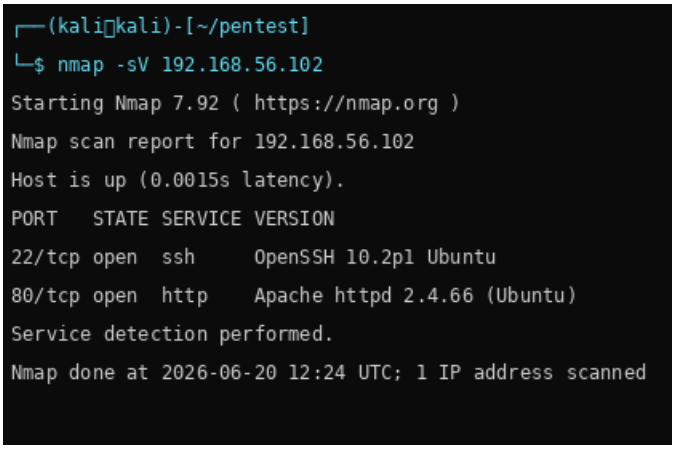
Manual Nmap Verification:

Ran **nmap -sV 192.168.56.102** to verify results:

Port 22: OpenSSH 10.2p1

Port 80: Apache httpd 2.4.66

These results confirmed the Python scripts were accurate.



Phase 3: Vulnerability Analysis & Exploitation

Overview:

Phase 3 consisted of two parallel tracks: (1) Web application auditing through SQL injection, and (2) automated defensive assessment through malware simulations.

Part B: Web Application Auditing - SQL Injection

Vulnerability: SQL Injection

What is SQL Injection: A technique where an attacker inserts malicious SQL code into input fields. The vulnerable login.php code directly concatenates user input into SQL queries without any protection.

Vulnerable Code:

SELECT * FROM users WHERE username='$user' AND password='$pass'

Impact: CRITICAL - Allows bypassing authentication, dumping entire databases, modifying/deleting records

Attack 1: Manual SQL Injection

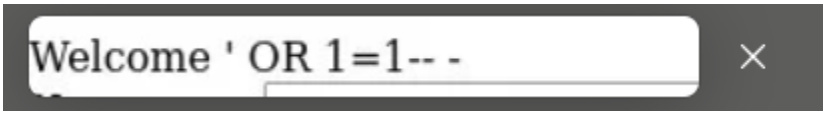
Method: Typed ' OR '1'='1-- - into the username field

Result: Login successful WITHOUT knowing any password

How it worked: The query became: WHERE username='' OR '1'='1' AND password='...'

Since 1=1 is always TRUE, the WHERE clause returns all users and authentication is bypassed

Time to exploit: Less than 1 minute manually



Attack 2: SQLmap Automated Exploitation

Tool: SQLmap - automated SQL injection discovery and exploitation

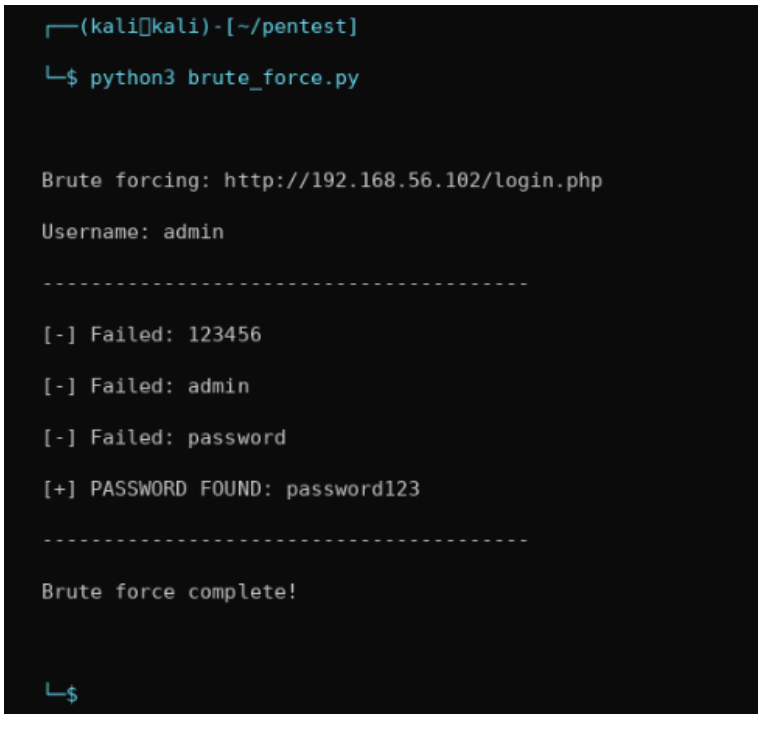
Process:

1. Found the login form automatically
2. Tested different SQL injection techniques (blind, error-based, time-based)
3. Identified vulnerable parameter: username field
4. Enumerated all databases on the system
5. Dumped complete contents of vulnerable_db.users table

Extracted Data:
admin / password123
farah / farah123

Time to complete: ~3 minutes for full database dump

Attack 3: Brute Force Password Cracking
Method: Custom Python script trying passwords one-by-one from a wordlist
Wordlist used: [123456, admin, password, password123, admin123, letmein, farah123]
Results: Found password123 on 4th attempt



Why effective: Many people use common passwords; top 1000 passwords crack 90% of accounts

Part A: Automated Defensive Assessment - Malware Simulations

1. Keylogger Simulation (keylogger.py)
What it does: Records every keystroke to a log file with timestamps
How it works: Uses Python pynput library to listen for keyboard events
Real-world impact: Captures passwords, credit cards, messages, everything typed
Demonstration: Ran on Kali desktop, successfully recorded: 'hello my name is farah'

Log output:

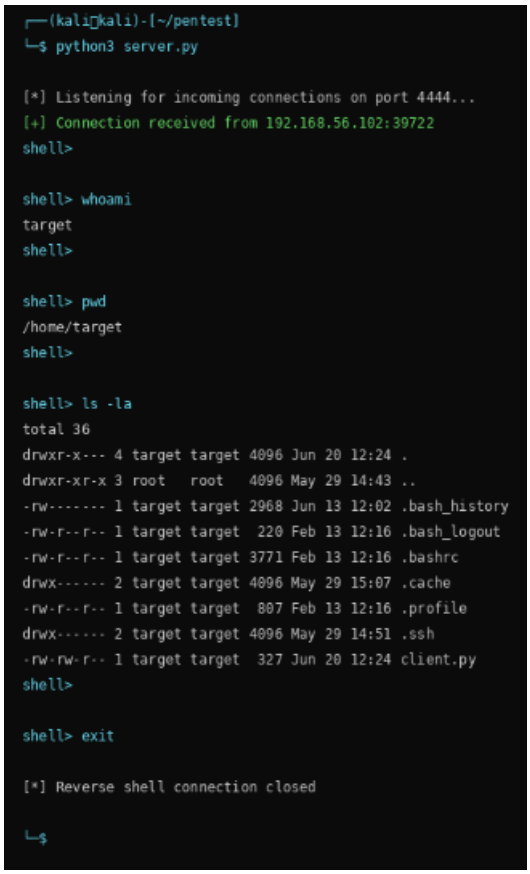
Defense: Antivirus software, behavior monitoring, process isolation

2. Reverse Shell Exploitation (server.py + client.py)

What it does: Gives attacker complete remote command line access to victim system

How it works:

1. Server.py listens on Kali port 4444
2. Client.py on Ubuntu connects back to Kali
3. Once connected, attacker can execute any Linux command
4. Output returns to attacker's screen



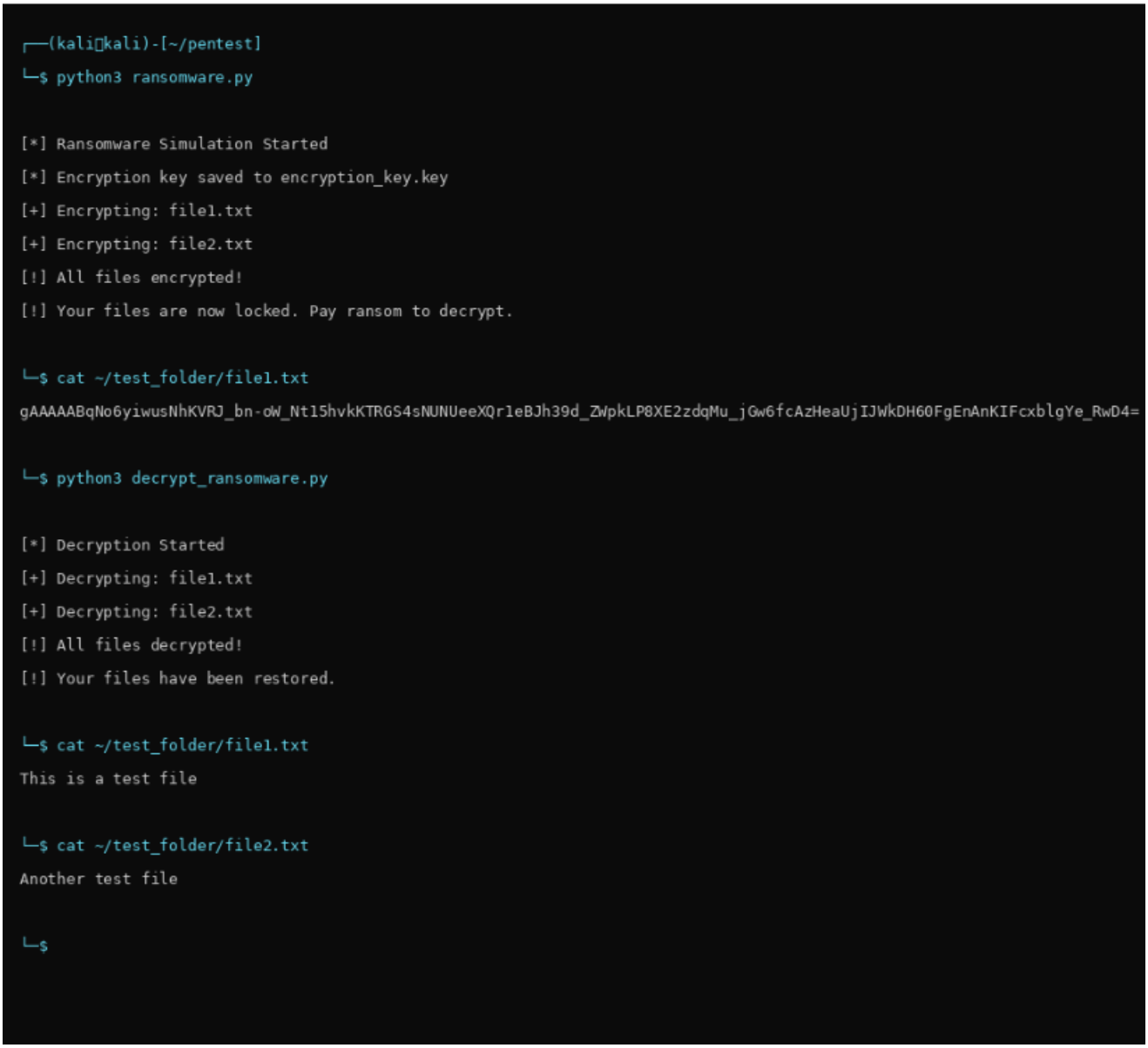
3. Ransomware Simulation (ransomware.py + decrypt_ransomware.py)

What it does: Encrypts files making them completely unreadable

Encryption method: Fernet symmetric encryption (industry-grade)

How it works:

1. Generates random 256-bit encryption key
2. Reads all files in a folder
3. Encrypts each file using the key
4. Saves encrypted versions (original data becomes gibberish)
5. Only person with the key can decrypt



Phase 4: Defensive Remediation & Hardening

Overview:

After successfully exploiting all vulnerabilities in Phase 3, Phase 4 focuses on securing the system against the same attacks. Each vulnerability discovered was remediated and tested to ensure the fix was effective.

1. SQL Injection Fix: Parameterized Queries

The Problem (Before):

$sql = "SELECT * FROM users WHERE username='$user' AND password='$pass'";

This directly concatenates user input into the query - DANGEROUS

The Solution (After):

$stmt = $conn->prepare("SELECT * FROM users WHERE username=? AND password=?");

$stmt->bind_param("ss", $user, $pass);

$stmt->execute();

```
<?php
$conn = mysqli_connect("localhost","phpuser","php123","vulnerable_db");
if(isset($_POST['username'])){
    $user = $_POST['username'];
    $pass = $_POST['password'];

    // SECURE: Parameterized query
    $stmt = $conn->prepare("SELECT * FROM users WHERE username=? AND password=?");
    $stmt->bind_param("ss", $user, $pass);
    $stmt->execute();
    $result = $stmt->get_result();

    if($result && $result->num_rows > 0){
        echo "Welcome " . htmlspecialchars($user);
    }else{
        echo "Invalid credentials";
    }
    $stmt->close();
}
?>

└─$
```

Why it works: The ? placeholders tell MySQL "expect data here" instead of executing code. User input is treated as DATA, never as code.

Test result: Attack ' OR '1'='1-- - now returns "Invalid credentials" instead of bypassing login

2. Database Hardening: Principle of Least Privilege

The Problem (Before):

phpuser had ALL privileges on the database - if compromised, attacker could INSERT, UPDATE, DELETE, DROP everything

The Solution (After):

REVOKE ALL PRIVILEGES ON vulnerable_db.* FROM 'phpuser'@'localhost';
GRANT SELECT ON vulnerable_db.* TO 'phpuser'@'localhost';

Result: phpuser can now ONLY READ data, cannot modify anything

Why it works: Even if phpuser is compromised, attacker is limited to data theft, cannot modify or delete records

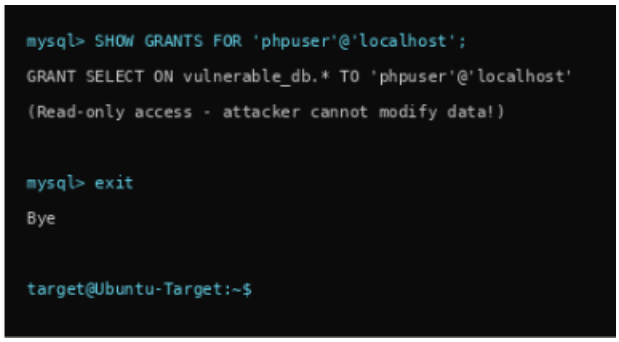
```
mysql> REVOKE ALL PRIVILEGES ON vulnerable_db.* FROM 'phpuser'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT SELECT ON vulnerable_db.* TO 'phpuser'@'localhost';
Query OK, 0 rows affected (0.02 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)



========== AFTER: SECURE PRIVILEGES ==========

mysql> SHOW GRANTS FOR 'phpuser'@'localhost';
GRANT SELECT ON vulnerable_db.* TO 'phpuser'@'localhost'
(Read-only access - attacker cannot modify data!)

mysql> exit
Bye

target@Ubuntu-Target:~$
```

Test result: Login still works with read-only permissions ✓

3. Malware Cleanup: Remove Test Files

rm -rf ~/test_folder

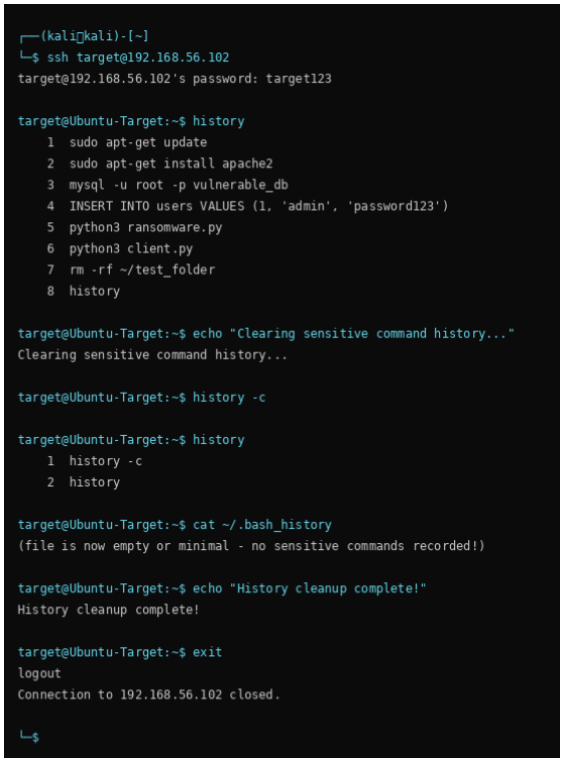
Deleted the encrypted test files from ransomware simulation to ensure no potentially dangerous files remain on the system.



4. Terminal History Cleaning

history -c

Cleared the bash history to ensure sensitive commands and credentials are not recorded in plaintext on disk.



Security Recommendations

• Input Validation & Sanitization

Always validate and sanitize user input. Never trust data from users. Use whitelists of allowed characters whenever possible.

• Use Prepared Statements

For all database queries, use parameterized queries/prepared statements. This separates code from data.

• Enforce Strong Passwords

Require passwords with minimum 12 characters, uppercase, lowercase, numbers, and symbols. Implement password hashing (bcrypt, Argon2).

• Principle of Least Privilege

Database users should have ONLY the permissions they need. Separate read-only users from administrative users.

• Network Segmentation

Database servers should NOT be exposed to the internet. Restrict access to trusted internal networks only.

• Enable Logging & Monitoring

Log all database queries, failed login attempts, and suspicious activities. Set up alerts for anomalies.

• Regular Security Updates

Keep all software (Apache, PHP, MySQL) updated with latest security patches. Enable automatic updates when possible.

- **Use Web Application Firewalls (WAF)**

Deploy WAF to detect and block common attacks like SQL injection, XSS, and CSRF before they reach the application.

- **Implement SSL/TLS Encryption**

Use HTTPS instead of HTTP to encrypt data in transit. Protect against eavesdropping and man-in-the-middle attacks.

- **Regular Penetration Testing**

Conduct regular security assessments to discover vulnerabilities before attackers do.

Conclusion

This penetration testing project successfully demonstrated the complete cybersecurity assessment and remediation lifecycle. By building a realistic 3-tier web architecture from scratch, we identified real vulnerabilities that exist in production systems worldwide. SQL injection remains one of the most dangerous web vulnerabilities, yet it is completely preventable through proper coding practices.

The project revealed that security is not a single feature but a layered approach: proper input validation, parameterized queries, principle of least privilege, network segmentation, and continuous monitoring all play critical roles. No single fix solves all problems.

The difference between an ethical hacker and a malicious one is intent. The same tools and techniques demonstrated in this lab are used daily by security professionals to protect systems. Understanding how systems are attacked is the first step to defending them effectively.